

Comparação de Técnicas de Machine Learning para Detecção de Fake News

Joyce da Costa Dias, Lucas Brandão Costa, Nathan Santos Ataliba, Vinícius Batista Ribas
& Jonas Lopes de Vilas Boas

Abstract— With the number of users on social media growing quickly, the dimension of information, including news, is taking place at an accelerated rate. In this context, the term fake news has risen, an expression largely used to describe and categorize misleading news. The objective of this article is to compare techniques of Machine Learning, combined with methods of Natural Language Processing, to identify which technique has the best metrics in fake news detection. For that, a database with 7.200 news, including 3.600 true news and 3.600 fake news, was used, applied to four Supervised Machine Learning algorithms: Support Vector Machine (SVM), Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), and eXtreme Gradient Boosting (XGBoost). These algorithms were chosen for their ability to handle complex textual data and detect subtle patterns in language. SVM excels at precisely separating true and false classes, while LSTM and GRU capture long-term dependencies in the text. XGBoost, on the other hand, is highly efficient in classifying complex data, identifying fake news based on textual features.

Index Terms— Fake News, GRU, LSTM, SVM, XGBoost

Resumo— Com o rápido crescimento do número de usuários nas redes sociais, a disseminação de informações, incluindo notícias, ocorre de maneira acelerada. Nesse contexto, emergem as chamadas *fake news*, termo amplamente utilizado para descrever e caracterizar notícias falsas. Este artigo tem como objetivo comparar técnicas de *Machine Learning*, aliadas a métodos de Processamento de Linguagem Natural, para identificar qual técnica oferece as melhores métricas na detecção de *fake news*. Para tanto, foi utilizada uma base de dados contendo 7.200 notícias, sendo 3.600 verdadeiras e 3.600 falsas, aplicadas a quatro algoritmos de Aprendizado de Máquina Supervisionado: Máquina de Vetores de Suporte (*Support Vector Machine* - SVM), Memória de Longo e Curto Prazo (*Long Short-Term Memory* - LSTM), Unidade Recorrente Fechada (*Gated Recurrent Unit* - GRU), e Aumento de Gradiente Extremo (*eXtreme Gradient Boosting* - XGBoost). Esses algoritmos foram escolhidos por sua capacidade de lidar com dados textuais complexos e detectar padrões sutis na linguagem. O SVM se destaca por separar com precisão classes verdadeiras e falsas, enquanto o LSTM e GRU capturam dependências de longo prazo no texto. Já o XGBoost é altamente eficiente na classificação de dados complexos, identificando *fake news* com base em características textuais.

Palavras Chave— Fake News, GRU, LSTM, SVM, XGBoost

I. INTRODUÇÃO

O termo *fake news* de acordo com o *site* [1], combina *fake* (falso) e *news* (notícias), referindo-se a notícias falsas

disfarçadas de verdade, frequentemente disseminadas em redes sociais e *sites*, influenciando opiniões, decisões e comportamentos. Esse fenômeno ganhou notoriedade nas eleições presidenciais dos EUA em 2016, segundo as informações contidas na notícia [2], sendo frequentemente associado ao impacto de informações enganosas em crises de saúde, política e economia, e provocando, em alguns casos, pânico e teorias da conspiração.

A disseminação rápida de *fake news* pelas redes sociais gera desinformação, impactando áreas críticas e contribuindo para a polarização social e distorção de processos eleitorais. Durante crises de saúde, como a pandemia de SARS-COV-2 (COVID-19), a disseminação de notícias falsas pode provocar comportamentos prejudiciais, como a recusa à vacinação ou a adoção de práticas arriscadas, fatos mencionados na notícia apresentada em [3]. Portanto, a detecção e o combate às *fake news* são fundamentais para a preservação da integridade das informações e para o funcionamento saudável das democracias. Atualmente, técnicas de detecção de *fake news* são aplicadas em várias áreas. Na saúde pública, ajudam a identificar notícias falsas sobre vacinação e doenças. Na mídia, buscam garantir a veracidade das informações compartilhadas. No *marketing*, monitoram informações que podem prejudicar campanhas e reputações [4].

Este trabalho visa comparar técnicas de Aprendizado Máquina, (*Machine Learning* - ML), principalmente técnicas supervisionadas como Máquina de Vetores de Suporte (*Support Vector Machine* - SVM), Memória de Longo e Curto Prazo (*Long Short-Term Memory* - LSTM), Unidade Recorrente Fechada (*Gated Recurrent Unit* - GRU), e Aumento de Gradiente Extremo (*eXtreme Gradient Boosting* - XGBoost), aliadas a métodos de Processamento de Linguagem Natural (PLN). O objetivo é identificar o modelo de melhor desempenho na detecção de notícias falsas. O diferencial do presente estudo em relação a outras comparações presentes na literatura é a inclusão do GRU, que, apesar de pouco explorado na detecção de *fake news*. O GRU se destaca por causa da sua capacidade de lidar com dados sequenciais e contextuais de forma eficiente, tendo uma arquitetura mais simples e menos exigente computacionalmente em comparação ao LSTM [5]. Além disso, ele supera as limitações de algoritmos, como SVM e XGBoost, em tarefas relacionadas a PLN [6].

Os métodos de ML e PLN foram escolhidos devido à sua capacidade de realizar análises rápidas em grandes quantidades de dados e à identificação de padrões ocultos que não são evidentes para humanos. O PLN é utilizado para analisar o significado das palavras, frases e textos, indo além do simples reconhecimento de palavras para compreender o que elas representam no contexto. Além disso, os modelos de ML podem ser continu-

amente atualizados e treinados com novos dados, permitindo que se adaptem às novas estratégias de desinformação. Essas técnicas são eficazes em distinguir entre notícias verdadeiras e falsas, enquanto o PLN possibilita que os algoritmos lidem de maneira eficiente com a linguagem humana. O SVM é eficiente na separação de classes. O LSTM e GRU capturam relações de longo prazo no texto e o XGBoost se destaca pela robustez e rapidez na classificação. Com isso, espera-se que o estudo contribua com uma análise robusta do GRU e de sua viabilidade para detecção de *fake news*. Para avaliar essas técnicas, utilizamos métricas de Acurácia, Precisão, *Recall*, F1-Score e Matriz de Confusão, que oferecem uma visão abrangente da performance.

O restante do trabalho está organizado como descrito a seguir. Na Seção II., será apresentado conceitos e teorias que sustentam o estudo, detalhando as técnicas de ML e explicando os algoritmos de aprendizado supervisionado empregados: SVM, XGBoost, LSTM e GRU. Na Seção III., será explorado estudos anteriores que utilizaram técnicas de ML para detecção de *fake news*. Na Seção IV., são detalhados os processos para a implementação do experimento. Na Seção V., são apresentados e analisados os resultados obtidos para cada algoritmo, utilizando métricas de desempenho e matrizes de confusão para uma avaliação mais precisa. Na Seção VI., são discutidos os resultados em profundidade, explicando o desempenho superior do SVM e debatendo as vantagens e limitações das demais abordagens analisadas. Na Seção VII., são resumidos os principais pontos do estudo dos algoritmos. Também reflete sobre as limitações do trabalho e propõe perspectivas futuras.

II. FUNDAMENTAÇÃO TEÓRICA

Nesta seção, são abordados os métodos, técnicas, algoritmos e métricas empregadas no desenvolvimento das comparações entre as abordagens propostas.

A. Processamento de Linguagem Natural

O Processamento de Linguagem Natural é uma subárea da Inteligência Artificial (IA) que se concentra na interação entre computadores e humanos por meio da linguagem natural. O principal objetivo do PLN é possibilitar que os computadores compreendam, interpretem e gerem texto ou fala de forma intuitiva e natural para os usuários. Essa capacidade de processar linguagem natural é crucial para uma ampla gama de aplicações, que vão desde assistentes virtuais até sistemas de tradução automática. No presente trabalho, o uso do PLN foi essencial para que as técnicas aplicadas compreendessem os textos. Algumas técnicas utilizadas no PLN serão discutidas a seguir.

a.1. Tokenização

De acordo com [7], a tokenização é o processo de dividir um texto em unidades menores, chamadas *tokens*, que podem corresponder a palavras, frases ou até mesmo caracteres. Esse processo foi essencial para os algoritmos de SVM, LSTM, GRU e XGBoost utilizados neste trabalho, pois facilitou a análise e manipulação dos dados textuais do *dataset*. Ao transformar o texto em *tokens*, os algoritmos puderam interpretar melhor o conteúdo e identificar padrões de forma mais eficaz

a.2. Vetorização

Baseado em [8] a vetorização é uma técnica que transforma palavras ou frases em vetores numéricos, permitindo que algoritmos de aprendizado de máquina, que só aceitam dados numéricos, possam processar as representações numéricas dessas informações textuais. No trabalho, a técnica de *Term Frequency-Inverse Document Frequency* (TF-IDF) foi utilizada para converter as notícias em vetores numéricos. Esses vetores foram utilizados como entrada para os algoritmos como SVM e XGBoost, permitindo que eles identificassem padrões entre notícias verdadeiras e falsas e realizassem a classificação com base nessas representações.

B. Aprendizado de Máquina

Do inglês *Machine Learning*, é uma subárea da IA que permite que os sistemas aprendam a partir de dados, fazendo previsões ou tomando decisões sem a necessidade de instruções programadas explicitamente. Como o próprio nome sugere, as técnicas de aprendizado de máquina focam em como capacitar as máquinas em aprender com base em um conjunto de dados, utilizando métodos de inferência indutiva para gerar resultados a partir desse aprendizado [9]. Uma das abordagens mais utilizadas será discutida a seguir.

b.1. Aprendizado Supervisionado

Neste método, o modelo é treinado com um conjunto de dados de entrada (X) e um conjunto de dados de saída (Y) rotulados, onde cada entrada está associada a uma saída correspondente. O objetivo é aprender uma função que relacione entradas a saídas. A Figura 1 exemplifica o modelo de Aprendizado Supervisionado.

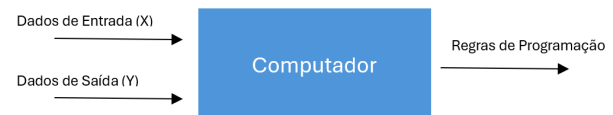


Fig. 1. Diagrama do processo de aprendizado de máquina supervisionado.

Neste trabalho, comparamos quatro técnicas de aprendizado de máquina supervisionado para identificar as abordagens mais eficazes na detecção de *fake news*. O aprendizado supervisionado é vantajoso nessa tarefa de acordo com [10], pois permite o uso de dados rotulados, proporcionando exemplos claros de notícias verdadeiras e falsas. Além disso, os modelos supervisionados oferecem maior precisão em classificação, têm resultados mais facilmente interpretáveis, são adaptáveis a novos dados e oferecem uma variedade de algoritmos que podem ser testados e comparados para encontrar a melhor abordagem.

C. Forma de incremento de aprendizagem

Forma de incremento de aprendizagem é relacionado na forma como o processo de aprendizado é alimentado com os dados de treinamento, variam entre dois tipos, são eles:

- *Batch (offline)*: O modelo é treinado em um lote completo de dados de uma só vez e o modelo não é atualizado com novos dados até que um novo treinamento seja iniciado manualmente.

- Fluxo contínuo (*online*): O modelo é treinado continuamente conforme novos dados chegam, ajustando seus parâmetros. O modelo é atualizado em tempo real, sem a necessidade de treinar tudo novamente.

D. Modo de aprendizagem

Modo de aprendizagem é relacionado sobre como o algoritmo aprende e faz previsões a partir dos dados. O modo de aprendizagem é dividido em dois tipos:

- Baseado em instâncias (*instance-based*): Quando uma nova entrada precisa ser classificada, o algoritmo compara essa entrada com todos os exemplos que ele já memoriza. Ele escolhe os exemplos mais semelhantes (instâncias) e baseia sua decisão neles.
- Baseado em modelos (*model-based*): O algoritmo aprende padrões e relações nos dados durante a fase de treinamento. Quando uma nova entrada chega, ele utiliza o modelo já treinado para fazer previsões rapidamente. Para as 4 técnicas (SVM, GRU, XGBoost e LSTM) utilizamos este modelo.

E. Métricas utilizadas nos resultados

Para avaliar a qualidade das técnicas utilizamos as métricas de Acurácia, *Recall*, Precisão e F1-Score.

A Acurácia mede o desempenho geral (tanto verdadeiros positivos quanto verdadeiros negativos).

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN} \quad (1)$$

A métrica de *Recall* mede a capacidade de encontrar todas as instâncias positivas.

$$\text{Recall} = \frac{VP}{VP + FN} \quad (2)$$

A Precisão mede a qualidade das previsões positivas.

$$\text{Precisão} = \frac{VP}{VP + FP} \quad (3)$$

A métrica F1-Score é a média harmônica entre a precisão e o *recall*.

$$\text{F1-Score} = 2 * \frac{\text{Precisão} * \text{Recall}}{\text{Precisão} + \text{Recall}} \quad (4)$$

Para analisar esses resultados de forma visual, usamos a Matriz de Confusão. Ela é uma tabela, como apresentada na Tabela I, que mostra as previsões do modelo em relação aos valores reais, permitindo identificar o número de acertos e erros para cada classe.

TABELA I
MATRIZ DE CONFUSÃO

		Classificação Real	
		Negativo	Positivo
Classificação Predita	Negativo	<i>VN</i>	<i>FP</i>
	Positivo	<i>FN</i>	<i>VP</i>

onde:

- Verdadeiros Positivos (*VP*): são os casos em que o modelo prevê as notícias corretamente como verdadeiras.
- Verdadeiros Negativos (*VN*): são os casos em que o modelo prevê as notícias corretamente como falsas.
- Falsos Positivos (*FP*): são os casos em que o modelo classificou as notícias erroneamente como positiva.
- Falsos Negativos (*FN*): são os casos em que o modelo classificou as notícias erroneamente como negativa.

F. Técnicas de Aprendizado Supervisionado

No contexto da detecção de *fake news*, diversas técnicas de aprendizado supervisionado têm sido empregadas. As técnicas comparadas nesse trabalho serão discutidas a seguir.

f.1. Support Vector Machine

O algoritmo SVM busca identificar a melhor maneira de dividir os dados em classes distintas, utilizando um hiperplano como fronteira de decisão. Esse hiperplano determina para qual grupo uma amostra será atribuída com base na análise dos dados. Inicialmente, o SVM define um hiperplano que maximiza a margem, ou seja, a maior distância possível entre o hiperplano e os pontos de dados mais próximos de cada classe. Esses pontos específicos são conhecidos como vetores de suporte, e desempenham um papel essencial no ajuste do modelo para maximizar a separação entre as classes.

O processo de cálculo do hiperplano é realizado internamente pelo algoritmo por meio de métodos de otimização baseados em funções de perda e restrições. Um parâmetro crucial nesse ajuste é o coeficiente de penalização *C*, que regula o equilíbrio entre a maximização da margem e a tolerância a erros de classificação. Valores elevados de *C* resultam em um modelo mais rígido, focado em minimizar erros, possivelmente às custas de uma margem mais estreita. Por outro lado, valores mais baixos de *C* permitem margens mais amplas, o que pode proporcionar maior flexibilidade ao lidar com ruídos nos dados.

Outra característica fundamental do SVM é o uso de funções de núcleo (*kernel*), que transformam os dados para uma dimensão superior, possibilitando a separação das classes em casos onde não é viável traçar um hiperplano linear. Essa transformação permite que o modelo lide com conjuntos de dados mais complexos, nos quais as classes não podem ser separadas por uma única fronteira linear.

Após o treinamento do modelo, a etapa de inferência utiliza o hiperplano obtido para classificar novas amostras. Esse processo determina a classe de cada dado com base na posição relativa em relação ao hiperplano, garantindo que a separação seja realizada com base na maximização da margem previamente calculada. Essas informações foram retiradas de [11].

Por fim, a Figura 2 ilustra graficamente a definição de um hiperplano para separar duas classes, evidenciando a importância dos vetores de suporte e da margem máxima no funcionamento do algoritmo.

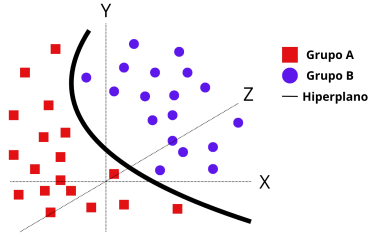


Fig. 2. Hiperplano que separa duas classes.

f.2. Xtreme Gradient Boosting

Um algoritmo de aprendizado de máquina baseado em árvores de decisão, otimizado pela técnica de *boosting em gradient descent*, técnica que combina vários modelos de árvores de decisão. A árvore de decisão é uma estrutura hierárquica que organiza informações em uma série de escolhas relacionadas. Cada nó da árvore representa uma decisão única, exemplificado na Figura 3.

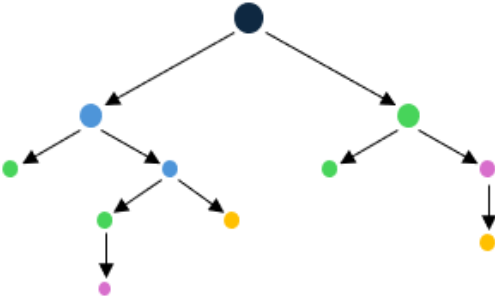


Fig. 3. Árvore de Decisão.

A Figura 3 foi utilizada como referência para o XGBoost, pelo fato do algoritmo utilizar como base a construção de múltiplas árvores de decisão em sequência para minimizar os erros no seu treinamento. No XGBoost, o modelo começa com uma previsão inicial, que é gerada automaticamente ao criar o modelo com XGBClassifier. Essa previsão inicial usa uma estimativa simples (como a média dos rótulos do conjunto de dados) e é ajustada iterativamente a cada nova árvore de decisão. O algoritmo compara essas previsões com os valores reais e calcula os erros, conhecidos como resíduos. Essa etapa é realizada internamente, sem necessidade de codificação manual. A partir daí, o XGBoost cria uma nova árvore de decisão que tenta corrigir esses erros, prevendo os resíduos das previsões anteriores. Esse processo é repetido em várias iterações, controladas pelo parâmetro que define o número de árvores que serão criadas (*n-estimators*). Cada nova árvore é ajustada para melhorar as falhas das anteriores, o que contribui para uma previsão mais precisa. Conforme cada árvore é adicionada, as previsões do modelo são atualizadas automaticamente. No final, o XGBoost combina as contribuições de todas as árvores, utilizando o método de que realiza a previsão final dessas árvores. O algoritmo também usa técnicas de regularização, controladas pelo parâmetro de profundidade máxima de cada árvore (*max-depth*), para evitar que o modelo "aprenda demais" com os dados de treino, ou seja, para prevenir o *overfitting*. Além disso,

o XGBoost utiliza um método de otimização conhecido como gradiente descendente para ajustar as árvores e reduzir os erros em cada iteração. Esse ajuste é controlado pelo parâmetro *learning-rate*, que define a taxa de contribuição de cada nova árvore ao modelo. Em resumo, o XGBoost constrói diversas árvores sequencialmente, cada uma ajudando a corrigir os erros das previsões anteriores. No final, todas as árvores contribuem juntas para a previsão final do modelo. Informações do XGBoost foram retiradas de [12].

f.3. Long Short-Term Memory

Uma arquitetura de rede neural projetada para lidar com sequências de dados, como texto, retendo informações de longo prazo. Com capacidade de remover ou adicionar informações através do uso de verificação em unidades chamadas "portões". Estes portões possibilitam o ajuste dos pesos e também a separação da sequência quando a informação for desnecessária, possuindo quatro camadas de interação que definem sua ligação.

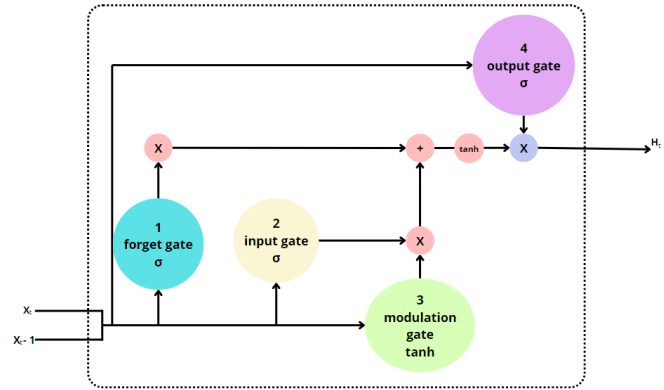


Fig. 4. Módulo repetições Redes LSTM.

Nas Redes LSTM, como mostrado na Figura 4, o processo começa com o *forget gate* (portão de esquecimento) (1), que decide quais informações da sequência anterior devem ser mantidas ou descartadas. Esse portão analisa os dados e atribui valores entre 0 e 1, onde 0 significa esquecida, enquanto 1 indica que será lembrada.

Em seguida, temos o *input gate* (portão de entrada) (2) e *modulation gate* (portão de modulação) (3), que definem quais novas informações serão armazenadas na célula de estado. O *input gate* usa uma função sigmoide (σ) para decidir quais valores serão atualizados. A função sigmoide, com sua saída entre 0 e 1, possibilita que o *input gate* module com precisão quais dados serão permitidos na célula, variando de uma retenção total, indicada pelo valor 1, até um bloqueio total, indicado pelo valor 0. Já o *modulation gate* gera novos valores candidatos a célula de memória usando a função tangente hiperbólica ($\tanh$). A função $\tanh$ gera valores entre -1 e 1, o que permite que o *modulation gate* represente uma gama mais ampla de informações, incluindo valores positivos e negativos, facilitando uma representação mais ampla de informações na célula.

O terceiro passo é a atualização da célula de estado, representado pelas três primeiras operações envoltas dos círculos ro-

sas de multiplicação (X) e soma (+). Onde as instruções de esquecimento e atualização são aplicadas, a célula de estado é modificada pela multiplicação e soma das informações passadas pelos portões anteriores, garantindo que apenas dados relevantes sejam mantidos ao longo da sequência. Por fim, no *output gate* (portão de saída) (4), o LSTM decide o que será enviado como saída, multiplicando as informações já atualizadas por uma função *tanh*, que ajusta os valores para o intervalo de -1 a 1, mantendo a estabilidade dos dados e destacando informações relevantes. Assim, apenas valores ajustados e selecionados são liberados para o próximo passo ou para a previsão final.

Esse processo se repete de acordo com o número de épocas, onde cada época corresponde a uma passagem dos dados de treinamento pelo modelo, ajustando seus pesos com base nos erros observados. Nas primeiras épocas, o modelo realiza grandes ajustes melhorando a precisão ao apreender padrões nos dados. A medida que as épocas aumentam, o modelo continua ajustando os pesos, porém os ganhos de precisão tendem a diminuir, pois ele já capturou maior parte dos padrões principais. A relação entre o número de épocas e o desempenho reflete o equilíbrio entre aprendizado suficiente e generalizado. Poucas épocas resultam em um modelo que não aprendeu o suficiente (*underfitting*), enquanto muitas épocas fazem com que o modelo memorize os dados de treinamento em vez de generalizar (*overfitting*). Informações do LSTM foram retiradas de [13].

f.4. Gated Recurrent Unit

A técnica GRU é uma arquitetura de Rede Neural Recorrente (RNN) projetada para lidar com dados sequenciais, sendo usada em aplicações como reconhecimento de texto através da fala, traduções de texto e previsões de séries temporais. As redes neurais recorrentes possuem um mecanismo de memória que permite à camada oculta (*hidden layer*) armazenar informações recebidas pela camada de entrada (*input layer*). Esses dados são ponderados com base no momento em que foram recebidos, de modo que informações mais recentes tem mais importância, enquanto dados mais antigos tendem a ser menos relevantes. O mecanismo de memória usa a repetição para reutilizar o estado oculto (*hidden state*), que representa os dados processados anteriormente.

No entanto, o GRU, assim como outras RNNs, enfrenta o problema comum de *vanishing gradient*. Ele ocorre quando o valor do gradiente se torna muito pequeno e irrelevante. O gradiente é uma medida que avalia quanto a saída de uma função muda em resposta a pequenas variações nas entradas. Durante a etapa de treinamento, ele é usado em conjunto com uma função de perda, que serve como um guia para ajustar os pesos da rede.

Semelhante ao algoritmo LSTM, o GRU simplifica sua estrutura utilizando dois portões, sendo o portão de redefinição (*reset gate*) e o portão de atualização (*update gate*). Essa simplificação proporciona mais eficiência sem comprometer sua capacidade de capturar dependências de longo prazo. A memória dessas dependências é mantida através do cálculo do GRU, que produz o estado oculto (*hidden state*), responsável por armazenar as informações relevantes da sequência.

No funcionamento do GRU, os dois portões - portão de redefinição (*reset gate*) e portão de atualização (*update gate*)

- definem quais informações devem ser mantidas ou descartadas. O processo começa com o portão de redefinição, que decide quais partes da informação passada devem ser esquecidas, útil para descartar dados irrelevantes para a previsão atual e focar em informações mais recentes. Em seguida, o portão de atualização controla quanto da informação deve ser retida no estado oculto, garantindo que as dependências de longo prazo não sejam perdidas durante o processamento de uma sequência. Conforme ilustrado na Figura 5, essa dinâmica é crucial para o funcionamento eficiente do GRU.

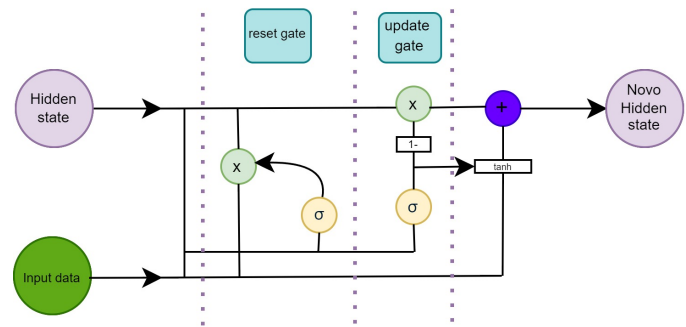


Fig. 5. Arquitetura simples de uma rede neural do tipo GRU.

Além disso, o GRU utiliza apenas o estado oculto para transmitir informações entre os portões, permitindo gerenciar simultaneamente as dependências de longo prazo e curto prazo. Os portões controlam o fluxo de informações, decidindo o que deve ser mantido ou descartado em cada etapa do processamento. Por isso, o GRU é importante para o processamento de grandes sequências de dados, pois captura informações sem perder o contexto das partes anteriores da sequência.

Durante o treinamento, os portões do GRU são ajustados para filtrar seletivamente qualquer informações irrelevantes. Geralmente, esses portões são vetores com valores entre 0 e 1 que são multiplicados com os dados de entrada ou o estado oculto. Um valor aproximado de 0 indica que os dados não são importantes e serão descartados, enquanto um valor aproximado a 1 indica que os dados são relevantes e serão usados no cálculo. Informações do GRU foram retiradas de [14].

III. TRABALHO RELACIONADOS

Nesta seção, são abordados os trabalhos relacionados que serviram de base para a realização da comparação das técnicas que foram utilizadas.

A. Comparison of Fake News detection using machine learning and deep learning techniques

No estudo apresentado em [15], é feita uma comparação entre os modelos de ML Naive bayes (NB), SVM e os modelos de *deep learning Neural Network with Keras* (NN-KERAS), LSTM, *Neural Network with TensorFlow* (NN-TF) na detecção de *fake news*. Todos os modelos citados foram avaliados com dois conjuntos de dados em inglês. Foram usadas quatro métricas: acurácia, precisão, *recall* e F1-Score. Os resultados mostraram que os modelos de *deep learning* apresentaram melhores resultados, o LSTM teve uma acurácia média de 94,21%, o NN-KERAS apresentou uma acurácia média de 92,99%, o NN-TF apresentou uma acurácia média de 91,16%, o SVM teve

uma acurácia de 88,24% e NB apresentou uma acurácia média de 75.1%.

B. *Processamento de Linguagem Natural - Detecção de Fake News*

No estudo apresentado em [16], foram comparados três algoritmos de *Machine Learning* em conjunto com o método de PLN: SVM Linear, SVM *Region-based Feature Blocks* (SVM RFB) e AdaBoost. A base de dados utilizada para o treinamento dos algoritmos foi o Fake.br Corpus, que contém 3600 notícias verdadeiras e 3600 notícias falsas. Para avaliar o desempenho dos algoritmos, foram empregadas as seguintes métricas: Precisão, *Recall*, F1-Score e Acurácia. Após a análise, o AdaBoost apresentou o melhor desempenho, alcançando 94% de Acurácia, 94% de Precisão, 96% de F1-Score e 92% de *Recall*.

C. *Performance Comparison of Machine Learning Classifiers for Fake News Detection*

No trabalho apresentado em [17] são comparados sete algoritmos de ML, sendo elas SVM Linear, *Logistic Regression*, *Decision Trees*, *Random Forest*, *Gradient Boosting*, XGBoost, Rede Neural, aplicados em três técnicas de PLN: *Countvectorizer*, TF-IDF, *Word Embedding*. Os dados processados consistem em cerca de 13.000 publicações extraídas de 244 sites. Após as análises, o SVM Linear com TF-IDF teve a melhor pontuação, apesar de ter um empate entre SVM Linear usando TF-IDF e a Rede Neural usando *Countvectorizer*, ambas com acurácia de 0.94, a Rede Neural apresentou ser mais lenta e complexa que o SVM Linear, tendo isso o SVM Linear mostrou melhor desempenho dentre os algoritmos mostrados.

D. *Classificação de Fake News a partir de Dados Textuais com Machine Learning*

Baseado na monografia [4], é apresentada a criação de um modelo de *Machine Learning* para identificar e classificar *fake news* sobre a vacinação, especialmente durante a pandemia de COVID-19 quando ocorreu a alta disseminação de desinformação nas redes sociais. Utilizando a metodologia CRISP-DM, o processo foi estruturado em seis fases, desde o entendimento do negócio até a implantação do modelo. Os dados foram coletados de redes sociais e projetos que resultaram em aproximadamente 115.597 entradas. Após a filtragem e pré-processamento, que envolveu a remoção de dados irrelevantes e duplicados, esse número foi reduzido para 42.665 entradas.

Além disso, o estudo aplicou técnicas de PLN para analisar os dados textuais, extraindo informações relevantes através de métodos como tokenização e remoção de *stop words*.

Para modelagem, foram testados vários modelos de ML, incluindo regressão logística, XGBoost, *Naive Bayes*, *Decision Tree*, AdaBoost e SVM, com a regressão logística se destacando por prever a probabilidade de uma notícia ser verdadeira ou falsa com base nas características textuais. O modelo demonstrou eficácia na identificação de padrões associados à desinformação sobre vacinação.

IV. METODOLOGIA

Esta seção apresenta as técnicas utilizadas para a detecção de *fake news* como Aprendizado de Máquina e de PLN. A análise

envolve métodos de PLN como tokenização e vetorização, aplicadas nos algoritmos SVM, XGBoost, LSTM e GRU, cada um aproveitando características distintas do PLN, como TF-IDF e *embeddings*. O processo inclui a coleta de dados do *dataset* Fake.br-Corpus¹, dividido igualmente entre notícias verdadeiras e falsas, e passa por etapas como divisão em treino e teste, otimização de hiperparâmetros com RandomizedSearchCV (para o XGBoost) e definição de parâmetros manuais (para LSTM e GRU). A comparação dos algoritmos é feita com base na métricas de acurácia, precisão, *recall* e *f1-score* que medem o desempenho das técnicas, fornecendo uma estimativa de qual técnica é mais eficaz.

Para o processamento dos dados, será utilizado métodos de PLN, que analisam e interpretam dados linguísticos, transformando a linguagem natural em um formato que os computadores podem processar. Entre os principais processos do PLN, destacam-se a tokenização, utilizada nos algoritmos SVM e XGBoost, e a vetorização, aplicada nos algoritmos LSTM e GRU.

A análise comparativa será feita utilizando a métrica de acurácia, que calcula a proporção de acertos (verdadeiros positivos e verdadeiros negativos) em relação ao total de amostras. Essa métrica permitirá avaliar a eficácia de cada técnica na detecção de *fake news*.

A escolha das técnicas foi baseada em análises de artigos relacionados ao tema, onde observamos que os algoritmos SVM, LSTM e XGBoost eram os mais frequentemente utilizados. Como diferencial, decidimos incluir o GRU, justamente por sua ausência em estudos similares. Nosso objetivo foi explorar e analisar o desempenho do GRU em comparação com essas técnicas mais comuns, avaliando seu potencial e investigando suas vantagens e limitações.

Para criação dos cenários de teste para todos algoritmos, foram necessárias algumas etapas, sendo elas:

- Coleta de Dados: Utilizou-se o *dataset* Fake.br-Corpus [18], devido a sua facilidade de acesso, compatibilidade com os objetivos da análise e ao fato de já estar pré-processado, esses fatores foram determinantes, considerando as limitações de tempo do escopo do projeto. O conjunto de dados contém 7200 notícias, sendo 3600 verdadeiras e 3600 falsas.
- Configuração das máquinas utilizadas: Foram utilizadas quatro máquinas para execução dos testes de cada algoritmo. É importante destacar que os testes realizados não tem como foco o desempenho das máquinas, mas sim os resultados obtidos nas métricas avaliadas.
 - Máquina 1: Processador Intel Core i7-1165G7 (11ª geração), 32 GB de RAM e gráficos Mesa Intel XE (TGL GT2).
 - Máquina 2: Processador Intel Core i5-12400F (12ª geração), 16 GB de RAM e gráficos NVIDIA RTX 4060 8GB.
 - Máquina 3: Processador Intel Core i7-8565U (8ª geração), com 16GB de memória RAM.

¹Dataset do GIT: <https://github.com/roneysco/Fake.br-Corpus>

– Máquina 4: Processador AMD Ryzen 7-4800H (4ª geração), 16GB de RAM e gráficos NVIDIA GTX 1650.

- **Vetorização/Tokenização:** Foi aplicada a Função TF-IDF nos modelos XGboost e SVM para realizar a vetorização e tokenização. Sendo que o SVM utiliza superfícies de decisão e o XGboost utiliza árvores de decisão. Essa função é composta por duas partes principais: o TF, que mede a frequência de uma palavra em um documento específico, e o IDF, que mede a importância da palavra em todo o corpus. Após isso é feito o cálculo do TF-IDF multiplicando o TF e IDF de cada *token* [19]. Já os modelos GRU e LSTM utilizam *embeddings*, onde cada palavra do texto é convertida em um vetor denso (*embedding*). A diferença entre os dois é quando a captura é feita, o GRU captura dependências em dados sequenciais, mas com menos complexidade.
- **Divisão dos Dados em Treino e Teste:** A divisão dos dados é essencial para avaliar a capacidade de generalização do modelo. O modelo é treinado em um subconjunto dos dados (conjunto de treinamento) e testado em outro (conjunto de teste). Isso foi feito em todos os algoritmos.
- **Treinamento do Modelo:** Utilizou-se a função *RandomizedSearchCV* para otimizar a busca pelos melhores hiperparâmetros do modelo XGboost. Essa abordagem permite testar várias combinações de hiperparâmetros e escolher a que maximiza a acurácia. Enquanto que o SVM utiliza hiperparâmetros padrão, e o LSTM e GRU foram setados manualmente.
- **Previsão com o Melhor Modelo:** Após o treinamento, o modelo otimizado é selecionado. A melhor combinação de hiperparâmetros é extraída, permitindo que o modelo obtenha o melhor desempenho nas previsões.
- **Avaliação das Técnicas:** para avaliar as técnicas, utilizamos as métricas de Acurácia (Equação 1), *Recall* (Equação 2), Precisão (Equação 3), F1-Score (Equação 4) e a Matriz de Confusão (Tabela I).

V. RESULTADOS

As pesquisas foram realizadas reunindo artigos da área e selecionando o *dataset* [20] disponível no GitHub, que contém as notícias ideais para o presente trabalho. Foram escolhidos quatro algoritmos de Aprendizado de Máquina para detecção de *fake news*, escolhemos os algoritmos GRU, LSTM, SVM e XGBoost. Cada um desses algoritmos trabalha de uma forma com exceção do GRU e LSTM que são similares. Os resultados obtidos com o *dataset* Fake.br-Corpus que tem os dados já pré-processados, o SVM alcançou uma acurácia com média de 95.8%, o XGboost alcançou uma acurácia de 95.0%, o LSTM alcançou uma acurácia de 92.8% e o GRU apresentou uma acurácia de 92.8%. É importante ressaltar que essas métricas foram alcançadas utilizando esse *dataset* e os resultados podem variar com o uso de outros conjuntos de dados, pois as características do *dataset*, como o formato e o pré-processamento dos

dados, tem um grande impacto no treinamento e desempenho dos algoritmos. LSTM e GRU são melhores ao lidar com dados temporais ou sequenciais, já o XGBoost lida melhor com dados tabulares e o SVM lida melhor com conjunto de dados menores e bem estruturados. A Tabela II apresenta os melhores valores das métricas obtidas nos testes, referente as 4 técnicas. A seguir vamos dar mais detalhes sobre os resultados das técnicas utilizadas.

TABELA II
TABELA DE COMPARAÇÃO DE MÉTODOS

	Acurácia	Precisão	Recall	F1-Score
SVM	0.958	0.967	0.948	0.958
XGBoost	0.950	0.955	0.944	0.949
LSTM	0.928	0.915	0.944	0.929
GRU	0.928	0.922	0.935	0.928

A. SVM

Foram feitos alguns testes utilizando o SVM e utilizando uma biblioteca chamada “*RandomizedSearchCV*” que é capaz de escolher o melhor parâmetro para treinar o modelo, onde se varia os parâmetros “C”, “gamma” e “kernel”. Geramos um gráfico comparando o *Recall*, F1-score, Acurácia e Precisão observando o *score* do algoritmo com os melhores parâmetros escolhidos pela biblioteca, a Figura 6, ilustra as métricas. Tendo como resultados: Acurácia de 95.8%, Precisão de 96.7%, *Recall* de 94.8% e F1-Score de 95.8%. Os resultados encontrados com a matriz de confusão são: 685 VP, 695 VN, 25 FP e 35 FN, como mostra a Figura 7.

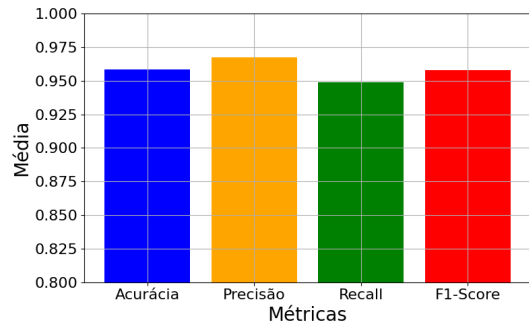


Fig. 6. Métricas do SVM.

Classe Real	Negativo	695	25
	Positivo	35	685
		Negativo	Positivo
		Classe Predita	

Fig. 7. Matriz Confusão do SVM.

B. XGBoost

Os resultados obtidos com a técnica de XGBoost foram satisfatórios. A função “RandomizedSearchCV” facilita a busca por melhores hiperparâmetros, aumentando a eficácia da técnica, variando os parâmetros “n-estimators”(quantidade de árvores), “max-depth”(profundidade da árvore) e “subsample”(evitar overfitting). As métricas calculadas apresentaram os seguintes valores: Acurácia de 95.0%, Precisão de 95.5%, *Recall* de 94.4% e F1-Score de 94.9%, os resultados foram plotados na Figura 8.

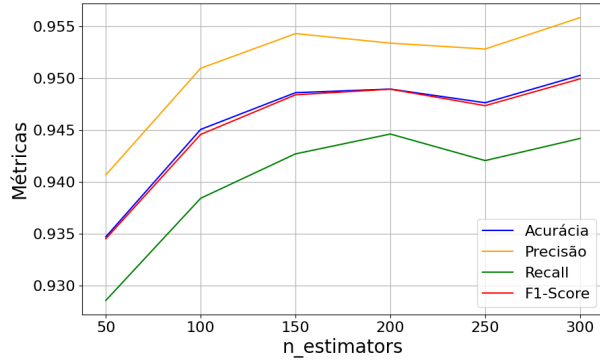


Fig. 8. Métricas de desempenho do XGboost em função do número de árvores (n-estimators).

Os valores obtidos na Matriz de Confusão são 692 verdadeiros positivos (VP), 681 verdadeiros negativos (VN), 35 falsos positivos (FP) e 32 falsos negativos (FN), como mostrado na Figura 9.

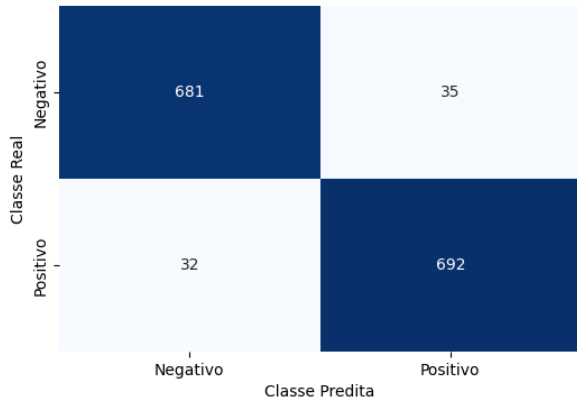


Fig. 9. Matriz de Confusão do XGboost.

C. LSTM

Após alguns testes com o LSTM percebemos variações em seus resultados, e devido a essa variação observadas, foi realizado uma média dos resultados gerados em 20 execuções distintas. Para evitar um *underfitting* ou um *overfitting*, foram usadas épocas variando entre de 5 a 55. Essa faixa foi escolhida para equilibrar o treinamento, evitando tanto o *underfitting* quanto o *overfitting*, além de reduzir a carga computacional.

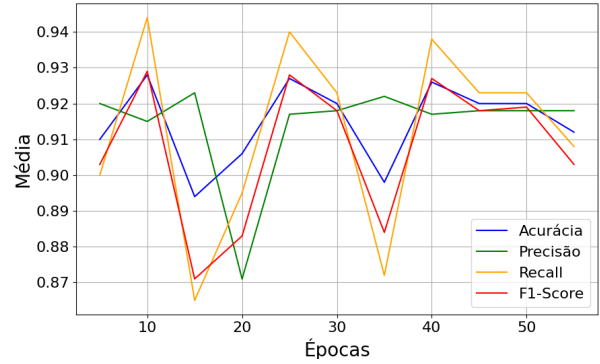


Fig. 10. Métricas de desempenho ao longo das épocas.

Como mostrado na Figura 10, podemos observar que o melhor desempenho ocorre com 10 épocas com valores médios de Acurácia de 92.8%, Precisão de 91.5%, *Recall* de 94.4% e F1-Score de 92.9%. Apesar da época 25 e 40 apresentarem valores semelhantes os valores de métricas de 10 épocas são ligeiramente melhores.

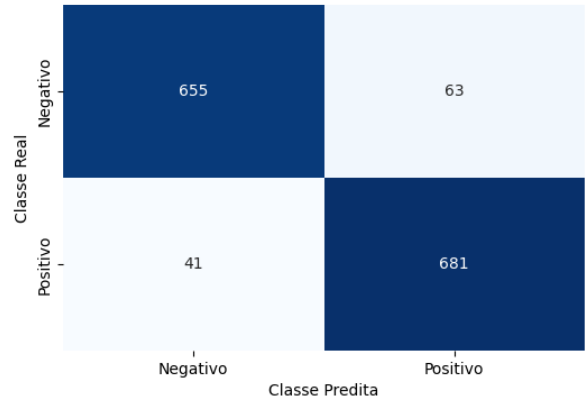


Fig. 11. Matriz Confusão LSTM.

Os resultados encontrados com a matriz de confusão em média são: 681 VP, 655 VN, 63 FP e 41 FN, como mostra a Figura 11.

D. GRU

Nos experimentos realizados com o algoritmo GRU, o principal objetivo foi avaliar sua eficácia em uma tarefa de classificação, testando o algoritmo 20 vezes em diferentes épocas para ter o melhor ponto de desempenho. Conhecido por sua habilidade em capturar padrões sequenciais complexos, o GRU foi avaliado com as métricas de Acurácia, Precisão, *Recall* e F1-Score. Conforme ilustrado na Figura 12, foi gerado um gráfico para observar o comportamento do modelo em função das épocas, visando a análise de como o algoritmo se comporta conforme o número de épocas aumenta.

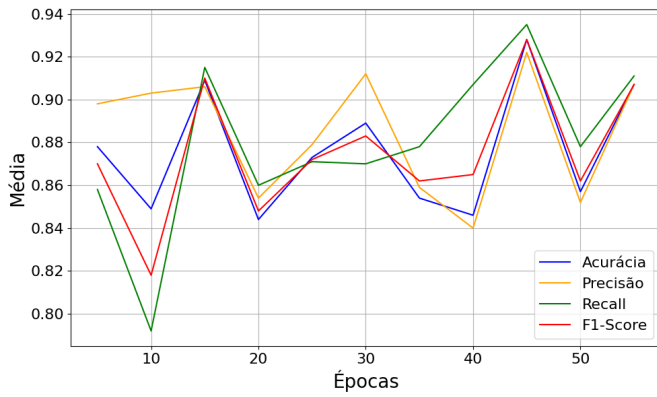


Fig. 12. Métricas de desempenho ao longo das épocas.

Com base na Figura 12, o melhor desempenho foi alcançado na época 45, onde o modelo obteve 92.8% de Acurácia, 92.2% de Precisão, 93.5% de *Recall* e 92.8% de F1-Score, demonstrando eficácia entre as métricas e comprovando a capacidade do GRU em capturar dependências de longo prazo sem perda significativa de informações importantes.

Além das métricas de desempenho apresentadas no gráfico, é possível analisar a matriz de confusão com a finalidade de avaliar a performance de um modelo de classificação, permitindo visualizar as previsões do modelo em comparação com os valores reais. Com base na Figura 13, podemos identificar tanto o número de acertos quanto de erros do modelo, como também compreender quais classes estão sendo corretamente previstas e quais estão sendo confundidas.

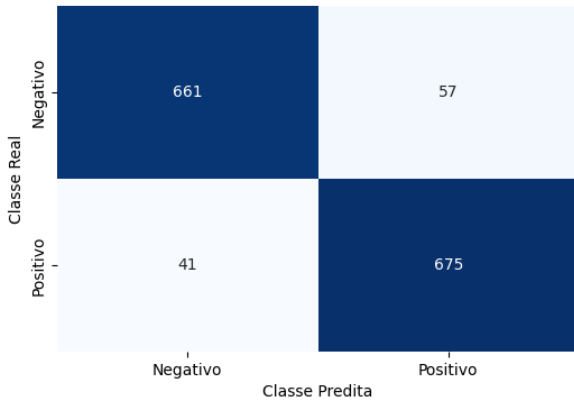


Fig. 13. Matriz Confusão GRU.

Assim, os resultados extraídos da matriz de confusão do GRU, como mostra a Figura 13, revelam que o modelo obteve 675 VP, 661 VN, 57 FP e 47 FN.

E. Comparação das Técnicas

Observamos na Figura 14 que o SVM obteve os melhores valores de desempenho entre as técnicas, seguido pelo XGBoost, LSTM e GRU.

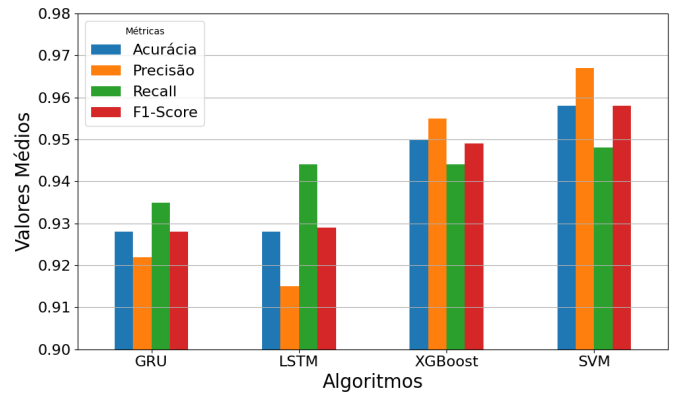


Fig. 14. Comparação dos desempenhos.

VI. DISCUSSÃO

Com base nos resultados, o algoritmo SVM obteve o melhor desempenho para dados sem estrutura sequencial (onde não há dependência temporal ou ordem a ser considerada), destacando-se por sua eficácia em conjuntos de dados com baixa dimensionalidade e necessidade de poucos dados para alcançar alta precisão. Por usar um classificador de margem para encontrar o hiperplano ótimo entre classes, ele é eficiente e oferece alta precisão sem precisar lidar com dependências temporais. Por essas características, o SVM é vantajoso em cenários onde a prioridade é obter precisão elevada com um modelo simples e eficiente.

Em contrapartida, os algoritmos LSTM e GRU são projetados para capturar dependências temporais e são mais adequados para dados sequenciais. Embora tenham apresentado bons resultados, exigiram mais ajustes de hiperparâmetros, aumentando a complexidade do código e o tempo de treinamento. O GRU, versão simplificada do LSTM, mostrou-se eficiente com menor uso de memória, o que é útil em ambientes com limitações de recursos computacionais.

O XGBoost também apresentou precisão próxima ao SVM, com acurácia de 95,0%, beneficiando-se de otimizações como a busca aleatória de hiperparâmetros. Contudo, exige mais recursos computacionais, especialmente em conjuntos de dados maiores.

Assim, os resultados indicam o SVM como a melhor escolha para problemas de classificação em dados tabulares com baixo custo computacional. Sua estrutura simples e demanda reduzida por ajustes de hiperparâmetros o tornam uma ferramenta eficaz para detecção de *fake news*. Por outro lado, LSTM e GRU são indicados para dados com dependências temporais, enquanto o XGBoost oferece um equilíbrio entre robustez e eficiência computacional.

Essa comparação ressalta a importância de selecionar o modelo adequado às características dos dados e restrições do projeto, promovendo uma abordagem estratégica para a detecção de *fake news*.

A. Vantagens e Desvantagens das Abordagens

Notamos que cada técnica possui seus prós e contras. As técnicas de LSTM e GRU, são excelentes para captar padrões em dados sequenciais e temporais, conseguindo manter informações relevantes em “memória” ao longo de várias etapas

do processo. Esse atributo permite que identifiquem padrões no texto que outras técnicas podem não captar. O GRU, em particular, apresenta uma estrutura mais simplificada que o LSTM, com menos portas de controle, o que o torna mais eficiente em termos de tempo de treinamento e uso de memória. Essa característica faz com que o GRU se destaque ao lidar com sequências longas e na detecção de *fake news*, pois consegue reter informações importantes de maneira autônoma, equilibrando precisão e eficiência computacional. No entanto, tanto o GRU quanto o LSTM exigem uma quantidade significativa de recursos computacionais, especialmente ao lidarem com grandes volumes de dados, tornando o treinamento mais demorado e caro.

O XGBoost é altamente eficiente para grandes conjuntos de dados tabulares (dados em tabela) e apresenta várias otimizações para acelerar o treinamento, como o uso da função `RandomizedSearchCV` para encontrar os melhores hiperparâmetros. Essas características permitem que o XGBoost seja rápido e preciso, especialmente ao lidar com dados estruturados. No entanto, ele pode se tornar computacionalmente caro quando o volume de dados é muito alto, o que aumenta significativamente o tempo de processamento e a necessidade de recursos computacionais mais robustos.

Já o SVM é geralmente eficiente para conjuntos de dados menores e oferece boa precisão em problemas de classificação linear e não linear. Ele possui a vantagem de diferentes opções de *kernel*, que permitem ajustar a complexidade do modelo conforme necessário. Por outro lado, o treinamento do SVM pode ser lento com grandes volumes de dados ou quando se utiliza um *kernel* que exige cálculos intensivos. Além disso, o SVM pode ser sensível a ruídos e valores atípicos, especialmente em conjuntos de dados desequilibrados.

B. Limitações e Desafios

Devido a limitações de tempo, decidimos trabalhar com um escopo reduzido, optando por apenas quatro técnicas. Essa escolha foi motivada pela necessidade de compreender do zero o funcionamento de cada abordagem, além de desenvolver as implementações utilizando ML e PLN. Como resultado, restringimos nossa análise ao modelo de aprendizado supervisionado e limitamos as métricas às mais conhecidas. Além disso, as técnicas de PLN foram limitadas a tokenização e Vetorização. As máquinas utilizadas nos testes, por não possuírem grande poder de processamento, impactaram a velocidade dos testes, resultando em testes mais lentos. Contudo, não foi realizado a medição do consumo de recursos computacionais, o que poderia fornecer uma análise mais detalhada sobre as limitações de *hardware*. Em relação ao *dataset*, todos os dados de treinamento eram textuais, o que restringiu a detecção de *fake news* apenas a esse tipo de informação.

C. Lições aprendidas

Durante o desenvolvimento deste trabalho, ganhamos uma compreensão significativa sobre os conceitos e práticas de ML e PLN. Aprendemos a trabalhar com diferentes tipos de algoritmos, entendendo conceitos importantes como *underfitting* e *overfitting*, que podem afetar o desempenho do modelo. Exploramos técnicas diferentes de PLN como vetorização e tokenização. Além disso, praticamos o uso de bibliotecas para

criar gráficos a fim de facilitar a compreensão e comparação dos resultados obtidos, permitindo análises mais claras e eficazes.

Também observamos que cada técnica de ML apresenta um desempenho específico de acordo com a quantidade de dados. Algumas técnicas precisam de mais dados e um tempo maior de treinamento para obter métricas mais altas, enquanto outras apresentam melhor eficiência em termos de processamento, exigindo menos recursos computacionais. Temos que considerar tanto a precisão quanto a eficiência da técnica de acordo com os objetivos.

Por fim, compreendemos que a escolha das métricas depende diretamente do tipo de dados e dos objetivos da análise. Métricas como Acurácia, Precisão, *Recall* e F1-Score, cada uma com suas particularidades, proporcionam diferentes perspectivas sobre a eficácia do modelo. Utilizando essas métricas em conjunto temos uma visão mais completa do desempenho geral de cada técnica.

VII. CONCLUSÃO

Durante o desenvolvimento deste trabalho, foram analisados quatro algoritmos de ML (SVM, XGBoost, LSTM e GRU) em um cenário comparativo de métricas. Na tabela II, o SVM destacou-se como o melhor desempenho geral, seguido pelo XGBoost, o LSTM e o GRU ficaram, respectivamente, em terceiro e quarto lugar, para o cenário/*dataset*, com diferenças mínimas entre si, com exceção da métrica de precisão em que o GRU ficou em terceiro e o LSTM ficou em quarto. Embora o SVM tenha apresentado melhores métricas, todos os algoritmos apresentaram um resultado robusto, acima de 90%.

Este trabalho contribui para comparar o desempenho do GRU, um algoritmo que, embora promissor, todavia, é pouco analisado em estudos diretos com modelos mais estabelecidos como SVM e XGBoost, que se destacam na maioria das pesquisas de desempenho. Ao adicionar o GRU em uma análise, esse estudo enriquece o entendimento do potencial desse algoritmo, mostrando que, embora o GRU não tenha superado o SVM em termos de métricas, ele ainda apresenta um desempenho competitivo, sugerindo que é uma alternativa viável em determinadas situações. O GRU não obteve um resultado superior, devido as características do *dataset* não serem as ideais, tendo em vista que o GRU tem seu desempenho projetado para lidar com *datasets* grandes e complicadas quantidades de dados. Apesar do GRU precisar de uma quantidade maior de dados que o SVM, não há uma quantidade exata que funcione em todos os casos, pois depende muito do tipo de dados (sequências ou temporais), da complexidade das relações temporais e da estrutura do problema.

A. Perspectivas Futuras

Para dar continuidade a esta pesquisa e explorar ainda mais o potencial dos algoritmos de ML, algumas perspectivas futuras incluem o aumento do cenário de testes, como a utilização de recursos computacionais, incorporando novas técnicas de ML como Regressão logística, *Naive Bayes* e KNN, além da exploração de outras abordagens de PLN como *Bag-of-Words* para avaliar diferentes métodos de pré-processamento. Um gráfico radar poderia ser utilizado para comparar o desempenho das técnicas, como complexidade, e poderia ser incluso o

tempo de treinamento e detecção, destacando suas vantagens e limitações.

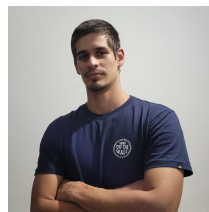
Também seria interessante em um trabalho futuro ampliar a base de dados e mudar o *dataset*, permitindo a análise de um maior volume e variedade de informações, além dos dados textuais. Essas futuras abordagens podem não apenas consolidar os resultados obtidos, mas também contribuir significativamente para o desenvolvimento de soluções mais eficazes em ML e PLN.

REFERÊNCIAS

- [1] Alynne Moreira Serra. “Fake News: Uma discussão sobre o fenômeno e suas consequências”. Em: (2018). URL: <https://monografias.ufma.br/jspui/bitstream/123456789/3466/1/ALYNNE-SERRA.pdf>.
- [2] Mike Wendling. *Como o termo 'fake news' virou arma nos dois lados da batalha política mundial*. URL: <https://www.bbc.com/portuguese/internacional-42779796> (acesso em 03/09/2024).
- [3] Edilson Rodrigues. *Desinformação e fake news são entraves no combate à pandemia, aponta debate*. URL: <https://www12.senado.leg.br/noticias/materias/2021/07/05/desinformacao-e-fake-news-sao-entreve-no-combate-a-pandemia-aponta-debate> (acesso em 07/09/2024).
- [4] Mel Iza Costa Augusto. “Classificação de fake news a partir de dados textuais com machine learning”. Em: (2023). URL: https://ric.cps.sp.gov.br/handle/123456789/15180?locale=pt_BR.
- [5] Pedro J. Freire, Sasipim Srivallapanondh, Bernhard Spinnler, Antonio Napoli, Nelson Costa, Jaroslaw E. Prilepsy e Sergei K. Turitsyn. “Computational Complexity Optimization of Neural Network-Based Equalizers in Digital Signal Processing: A Comprehensive Approach”. Em: *Journal of Lightwave Technology* (nov. de 2023). URL: <https://arxiv.org/pdf/2206.12191>.
- [6] Senlin Zhang, Yixing Wang, Meiqin Liu e Zhejing Bao. “Data-Based Line Trip Fault Prediction in Power Systems Using LSTM Networks and SVM”. Em: *IEEE Access* 6 (2018), pp. 7675–7686. DOI: 10.1109/ACCESS.2017.2785763.
- [7] Abid ALL Awan. *O que é tokenização?* URL: <https://www.datacamp.com/pt/blog/what-is-tokenization> (acesso em 18/09/2024).
- [8] Aprender Estatística Fácil. *O que é: Vetorização*. URL: <https://estatisticafacil.org/glossario/o-que-e-vetorizacao-entenda-o-conceito/> (acesso em 18/09/2024).
- [9] Victor Toledo Lupa Charleaux. *Machine Learning: o que é, como funciona e quais são os tipos de aprendizado de máquina*. URL: <https://tecnoblog.net/responde/machine-learning-o-que-e-como-funciona-e-quais-sao-os-tipos-de-aprendizado-de-maquina/> (acesso em 28/09/2024).
- [10] alteryx. *Aprendizado supervisionado vs. não supervisionado*. URL: <https://www.alteryx.com/pt-br/glossary/supervised-vs-unsupervised-learning> (acesso em 28/09/2024).
- [11] Daniel Gonçalves. “Inteligência Artificial”. Em: (2007). URL: <https://www.inf.ufpr.br/dagoncalves/IA07.pdf>.
- [12] Carlos Melo. *XGBoost: aprenda este algoritmo de Machine Learning em Python*. URL: <https://sigmoidal.ai/xgboost-aprenda-algoritmo-de-machine-learning-em-python/> (acesso em 28/09/2024).
- [13] Gustavo Cleiton Favero. “Modelo para predição dos ativos totais de uma cooperativa de crédito baseado em redes long short-term memory (LSTM)”. Em: (2019). URL: <https://rd.uffs.edu.br/handle/prefix/3319>.
- [14] Breno Phillip Mendes Baronte. “Redes Neurais Recorrentes Aplicadas à Predição de Valores de Ações”. Em: *Projeto de Graduação 1.1* (jan. de 2022). Universidade Federal do Estado do Rio de Janeiro (UNIRIO), Rio de Janeiro, RJ, Brasil, pp. 1–48.
- [15] M F Mridha, Ashfia Jannat Keya, Md Abdul Hamid, Muhammad Mostafa Monowar e Md Saifur Rahman. “A comprehensive review on fake news detection with deep learning”. Em: *IEEE Access* 9 (2021), pp. 156151–156170.
- [16] Guilherme Yujio Sakurai. “Processamento de Linguagem Natural - Detecção de Fake News”. Em: *Universidade Estadual de Londrina* (2019). URL: https://www.uel.br/cce/dc/wp-content/uploads/TCC_GUILHERME_SAKURAI.pdf.
- [17] N. Smitha e R. Bharath. “Performance Comparison of Machine Learning Classifiers for Fake News Detection”. Em: *2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA)*. Vol. 4. IEEE, jul. de 2020, pp. 696–700. DOI: 10.1109/icirca48905.2020.9183072. URL: <http://dx.doi.org/10.1109/ICIRCA48905.2020.9183072>.
- [18] Renato M. Silva, Roney L.S. Santos, Tiago A. Almeida e Thiago A.S. Pardo. “Towards automatically filtering fake news in Portuguese”. Em: *Expert Systems with Applications* 146 (2020), p. 113199. ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2020.113199>. URL: <http://www.sciencedirect.com/science/article/pii/S0957417420300257>.

- [19] Christopher D. Manning, Prabhakar Raghavan e Hinrich Schütze. *Introduction to Information Retrieval*. MIT Press, 2008. ISBN: 978-0-262-56329-0.
- [20] Rafael A. Monteiro, Roney L. S. Santos, Thiago A. S. Pardo, Tiago A. de Almeida, Evandro E. S. Ruiz e Oto A. Vale. “Contributions to the Study of Fake News in Portuguese: New Corpus and Automatic Detection Results”. Em: *Computational Processing of the Portuguese Language*. Springer International Publishing, 2018, pp. 324–334. ISBN: 978-3-319-99722-3.

AUTORES

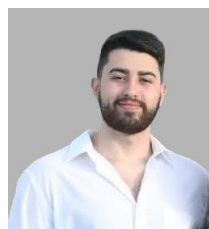


Nathan Ataliba é estudante de Engenharia de Computação do 10º período do INATEL- Instituto Nacional de Telecomunicações. Técnico em eletrônica com ênfase em Automação Industrial (2019) pela Escola Técnica de Eletrônica - ETE “FMC”. É Técnico de Certificação pela TechLab, atuando na homologação de produtos para

ANATEL. Tem interesse nas áreas de Engenharia de *Software*, Redes e infraestruturas e Tecnologia da Informação.



Joyce da Costa é estudante do 10º período de Engenharia de Computação no Instituto Nacional de Telecomunicações (Inatel). Atualmente, atua como *Quality Assurance*, contribuindo para a melhoria contínua dos processos de software. Além disso, possui interesse em aprofundar seus conhecimentos na área de Engenharia de *Software*, com ênfase em *Machine Learning*.



Lucas Brandão Costa é acadêmico do 10º período de Engenharia de *Software* no Instituto Nacional de Telecomunicações (Inatel). É Técnico em Eletrônica com ênfase em Telecomunicações (2018) pela Escola Técnica de Eletrônica – ETE “FMC” e tem interesse nas áreas de Engenharia de *Software*, Telecomunicações e Tecnologia da Informação.



Vinícius Batista é acadêmico do 10º período de Engenharia de Computação no Instituto Nacional de Telecomunicações (Inatel). Possui interesse na área de programação *back-end*, desenvolvimento de jogos e análise de dados.



Jonas Lopes de Vilas Boas é Engenheiro de Computação formado pelo Instituto Nacional de Telecomunicações (Inatel), MG, Brasil em 2016. Recebeu o título de Mestre em Ciências e Tecnologias da Computação pela Universidade Federal de Itajubá (UNIFEI), MG, Brasil em 2018, e o título de Doutor em

Telecomunicações pelo Inatel em 2023. É professor no Inatel e pesquisador na área de otimização de processos utilizando tecnologias emergentes como Internet das Coisas, Inteligência Artificial e Tecnologia de *Ledger* Distribuído.